My title

My name

October 27, 2026

1 Analysis

1.1 Description of missingness

```
n_participants <- nrow(copd)
n_variables <- ncol(copd)
missing_per_variable <- colMeans(is.na(copd)) * 100

min_missing <- sprintf("%.1f", min(missing_per_variable))
max_missing <- sprintf("%.1f", max(missing_per_variable))

pct_variables_lt5 <- sprintf("%.1f", mean(missing_per_variable < 5) * 100) # mean calcula la proporc
# equivalent: AA <- sprintf("%.1f", sum((missing_per_variable < 5))/n_variables * 100) # true=1, fal

pct_complete_cases <- sprintf("%.1f", mean(complete.cases(copd)) * 100)
pct_missing_total <- sprintf("%.1f", mean(is.na(as.matrix(copd))) * 100)

total_cells <- prettyNum(n_participants * n_variables, big.mark=",") # n_participants * n_variables
```

The subset used here included 342 participants and 45 variables. The range of missing data in the different variables went from 0.0% to 31.6%, with 68.9% of the variables having less than 5% of the data missing. Only 33.3% of the participants had complete data on all 45 variables. Overall, 5.4% of the information was missing; that is, 5.4% of the total of 15,390 cells (45 x 342 = 15,390) had missing data.

1.2 Multiple imputation

Apply multiple imputation to copd data using 100 imputations. Explain in a reasoned manner how you decided to set the values of the arguments method and predictorMatrix when using the mice() function. Show your code

```
ini <- mice(copd, maxit = 0)

meth <- ini$method
pred <- ini$predictorMatrix

meth["id"] <- "" # do not impute "id"
pred["id", ] <- 0 # we don't want "id" to be imputed
pred[, "id"] <- 0 # we don't want "id" to be predictor

# Fem les 100 imputacions:
imp <- mice(copd, method = meth, predictorMatrix = pred, m = 100, printFlag = FALSE, seed = 123)
```

Resposta: Hem mantingut el mètode i la matriu de prediccions amb l'excepció de l'id, ja que no es rellevant per al procediment. El valor de la 'm' és el nombre d'imputacions, printFlag es per mostrar en quina imputació està, que tampoc és important pels resultats, i la 'seed' és per a una facil replicació dels resultats.

1.3 Integration of multiple imputation in cluster analysis

Reproduce the original analysis in the paper using the miclust package to integrate multiple imputation in the cluster analysis. Apply the proposed framework using the same setting than in the original analysis.

To do that, follow these steps:

1. Read the help of the getdata() function in the miclust package. Then, accordingly, create a list of data.frames and pass it to the getdata() function. Give a detailed explanation of the data processing you did to get that list. Show your code.

```
resultat_mice <- complete(imp, action = "all", include = TRUE)
# action = "all" ja em posa les dades incials com a primer element

categoriques <- c(
  "sin11_v1", "iam_chv1", "sin15_v1", "sin7_v1", "sin8_v1",
  "cccs1_v0", "vtit_catv1", "urg_ambv0", "sin5_v1",
  "sin6_v1", "graudisn_v1"
)

resultat_mice <- lapply(resultat_mice, function(d) {
  d$id <- NULL
  d[categoriques] <- lapply(d[categoriques], function(x)
    as.numeric(as.character(x)))
  d
})

miclustdata <- getdata(resultat_mice) # adapta les dades al format de miclust
```

Resposta: Com 'getdata()' necessita una llista de dataframes, on el primer element conté les dades inicials i els següents les imputacions, hem cridat a la funció 'complete' que retorna just allò, amb "action='all'" i "include=TRUE" que és el que demana aquesta funció per a retornar el que volem. A més ens cal adaptar les variables categòriques com a numèriques, aleshores definim una funció que transforma els valors en text a valors numèrics, i aplicant la funció amb 'lapply()' amb una llista de les variables categòriques, aconseguim la llista de dataframes que demana la funció 'getdata()'.

2. Use the miclust() function to perform the integration of missing data in cluster analysis using the same settings as in the original analysis. Show your code and explain how and why you set all arguments in the miclust() function.

```
# Segon apartat:
copd_miclustdata <- miclust(data = miclustdata,search = "backward",
                            ks = 2:3,
                            distance = "manhattan",
                            centpos = "means",
                            initcl = "hc",
                            verbose = TRUE,
                            seed = 123)

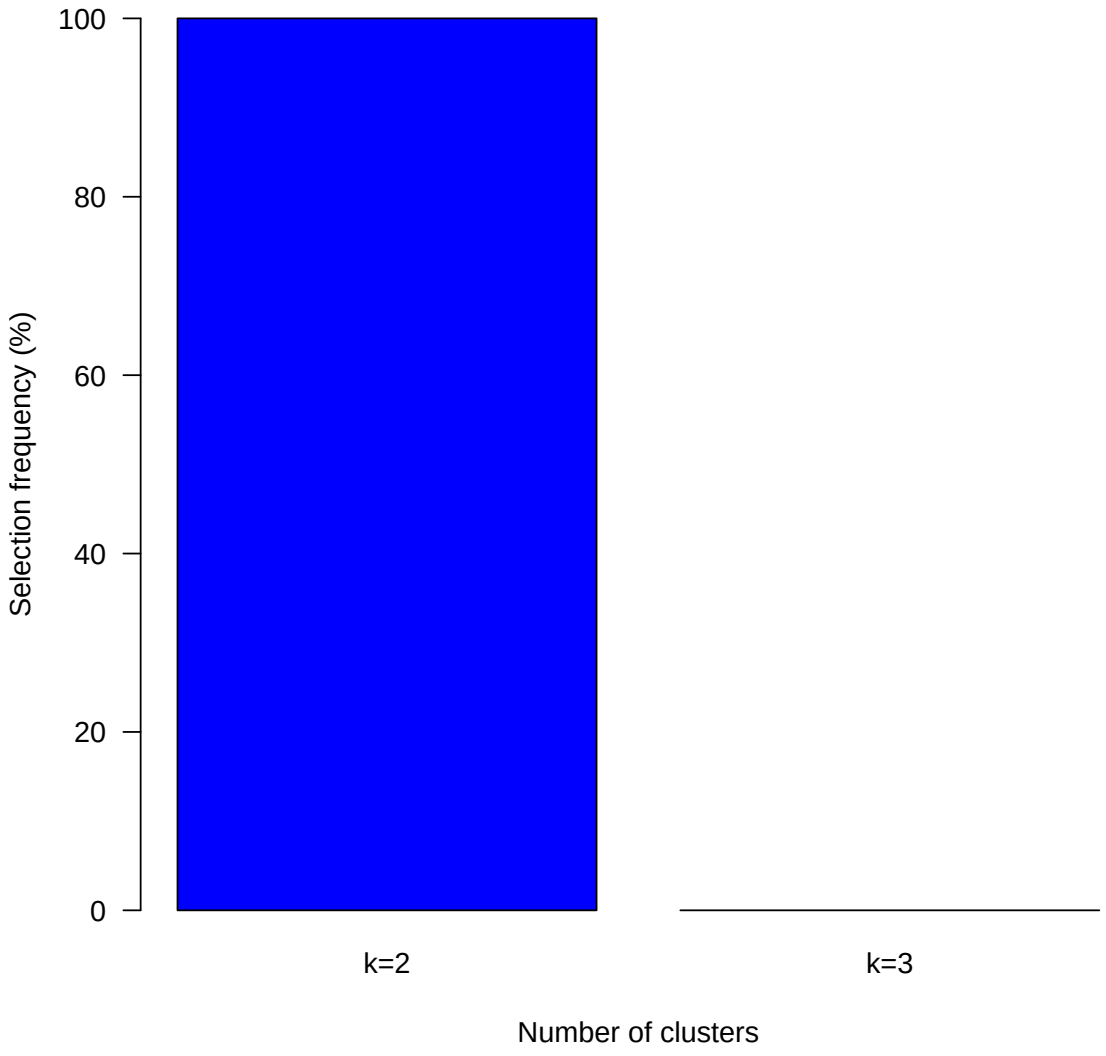
## . . . . imp   5 . . . . imp  10 . . . . imp  15 . . . . imp  20
## . . . . imp  25 . . . . imp  30 . . . . imp  35 . . . . imp  40
## . . . . imp  45 . . . . imp  50 . . . . imp  55 . . . . imp  60
## . . . . imp  65 . . . . imp  70 . . . . imp  75 . . . . imp  80
## . . . . imp  85 . . . . imp  90 . . . . imp  95 . . . . imp 100
##
##  Analysis done.
```

Resposta: Els elements escollits han estat els següents: 'backward', ja que és el que s'ens va instruir a classe 'ks = 2:3', l'article ja donava a entendre que un valor de clusters molt elevat careix de sentit 'manhattan', és l'opció per defecte 'means', estem utilitzant l'algorisme de k-means 'hc', hem decidit que l'inicialització sigui jeràrquica en lloc de a l'atzar per a millors resultats 'verbose = TRUE', d'aquesta manera mostrem el progrés d'aquest procés tan llarg 'seed = 123', per a una facil replicació dels resultats

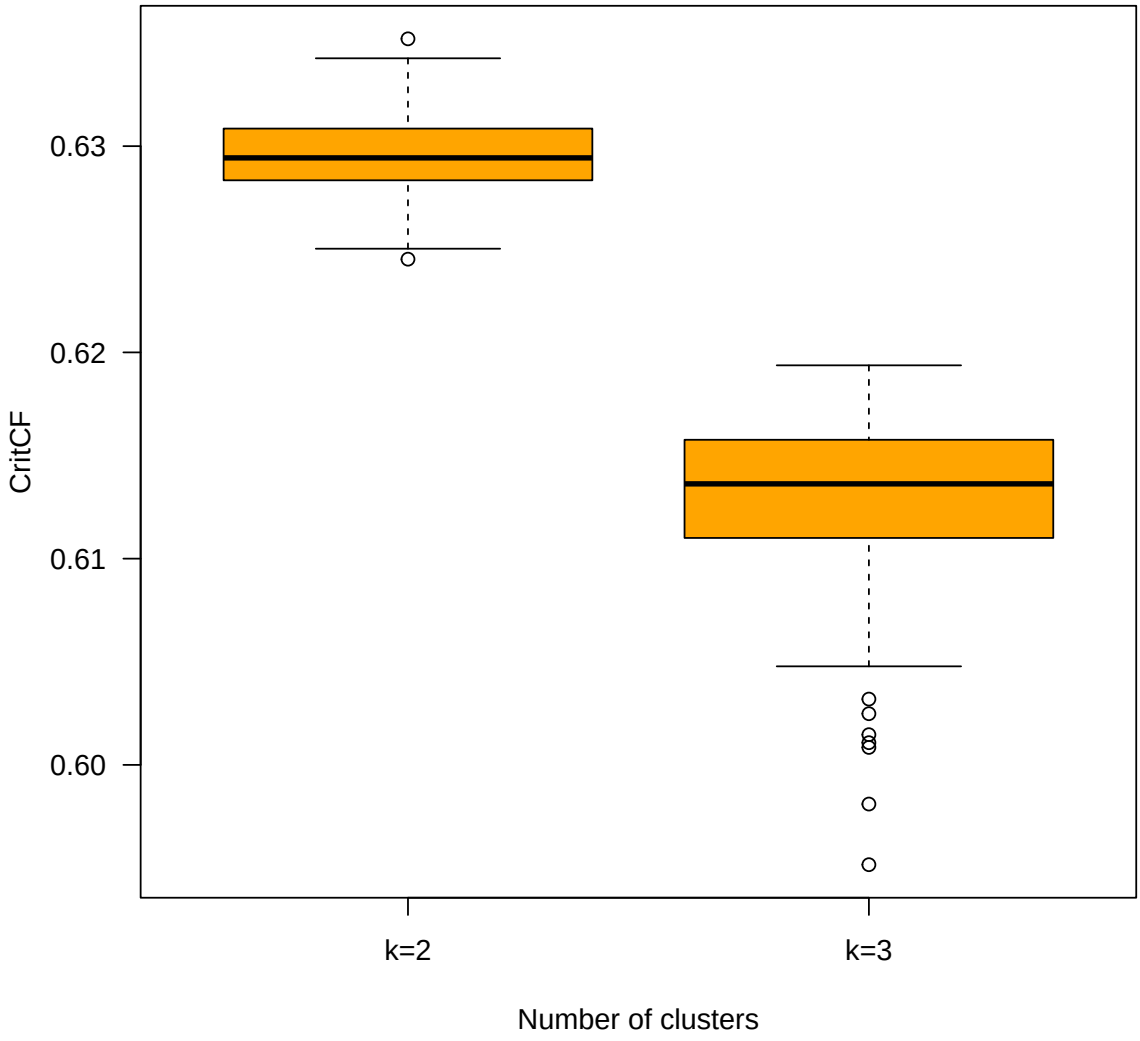
2 Results

Using plot() and summary() methods in the miclust package, provide plots1 and tables for the results similar than those shown in the original paper. Also, show a barplot for the percentage of times each variable is selected among the imputed data sets, in decreasing order2. Explain how you set all arguments in all functions and methods you use. Show your code.

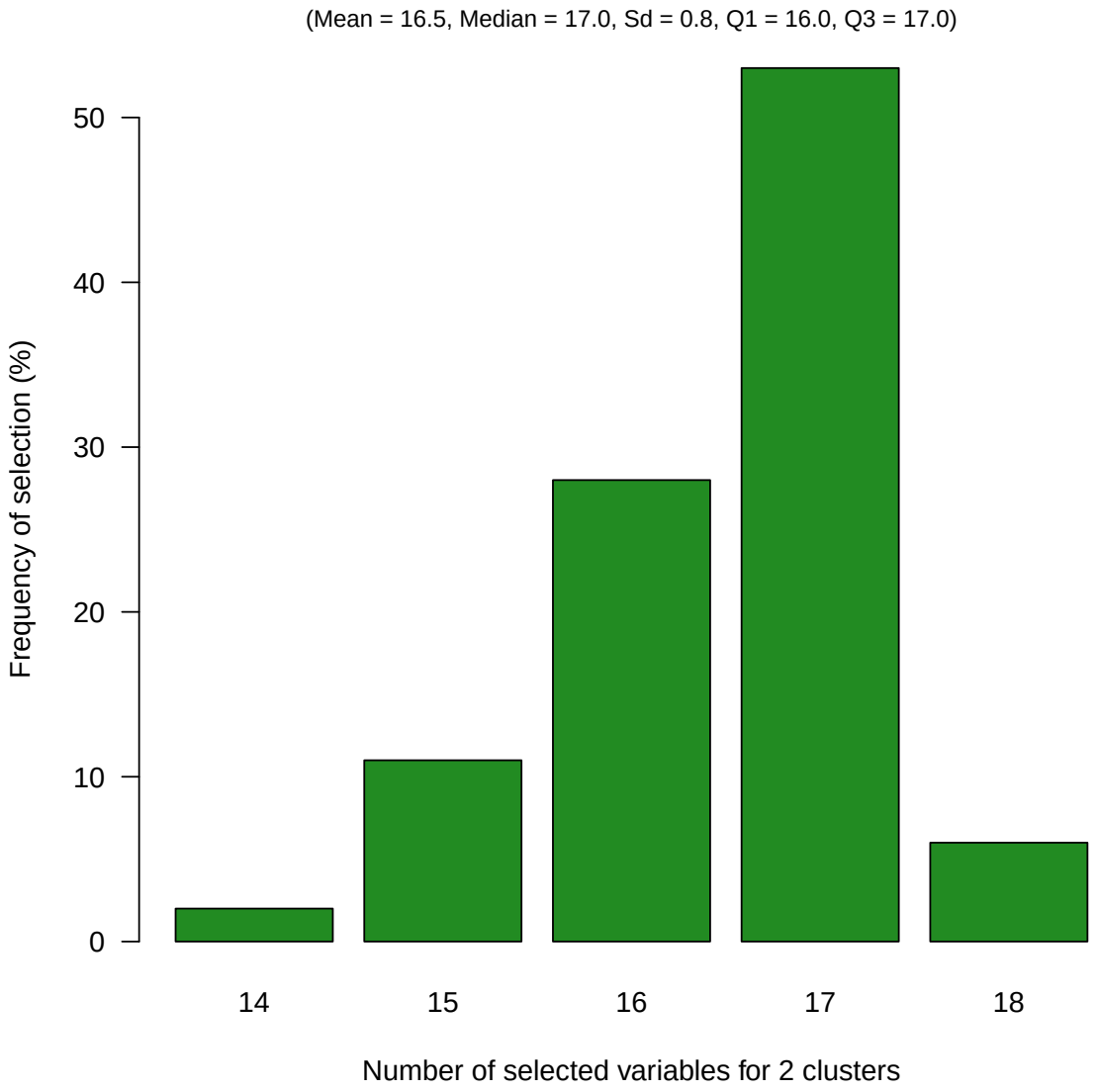
```
### plot results:
plot(copd_miclustdata, metric="nclfreq", col.nclfreq = "blue")
```



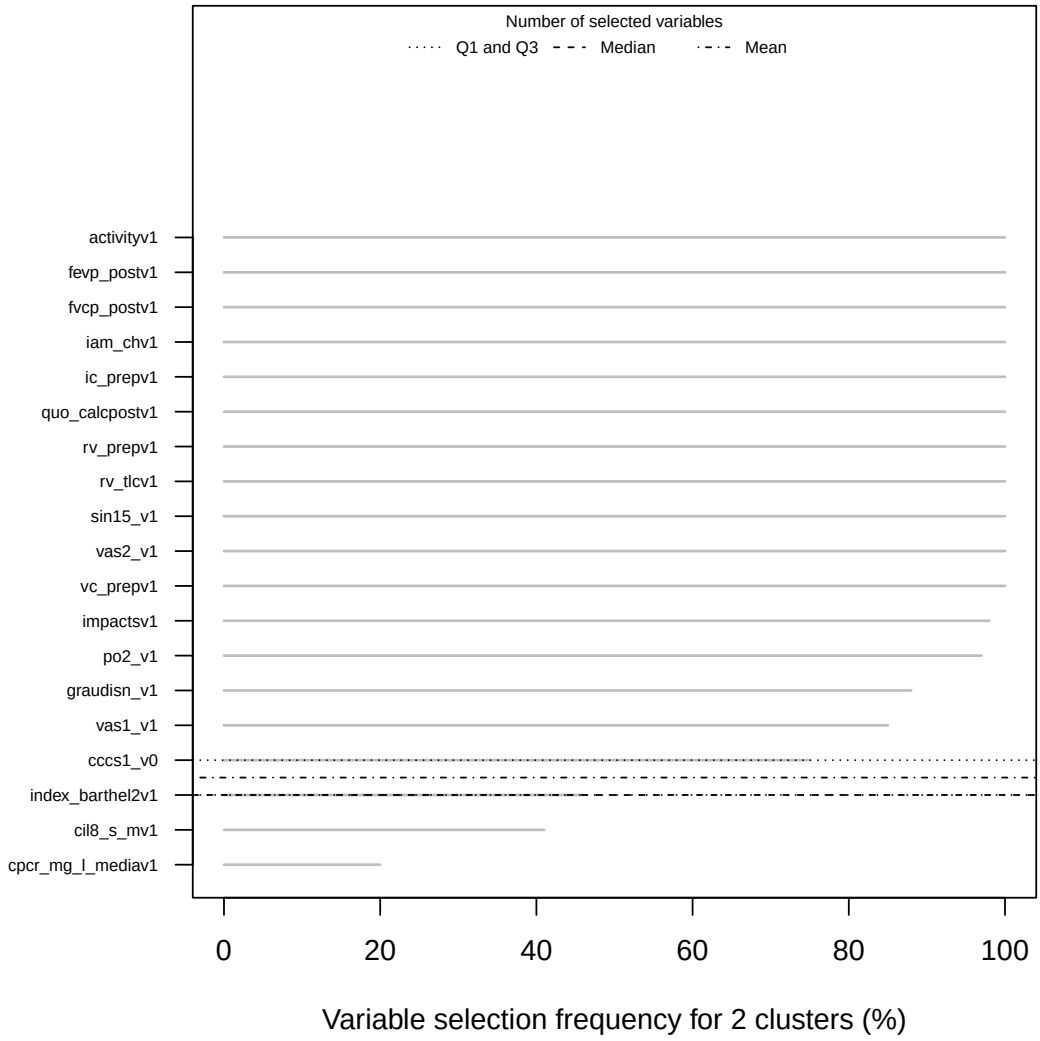

```
plot(copd_miclustdata, metric="critcf", col.critcf = "orange")
```




```
plot(copd_miclustdata, metric="nvarfreq", col.nvarfreq = "forestgreen")
```




```
plot(copd_miclustdata, metric="varsel", col.varsel = "gray")
```



Resposta: Els plots només calia definir els colors per a mostrar-ho

```
res2 <- summary(copd_miclustdata, k = 2, quantilevars = 0.5)
res3 <- summary(copd_miclustdata, k = 3, quantilevars = 0.5)

resum_prob <- rbind(res2$allocationprobabilities, res3$allocationprobabilities)

rownames(resum_prob) <- c("k = 2, Cluster 1",
                          "k = 2, Cluster 2",
                          "k = 3, Cluster 1",
                          "k = 3, Cluster 2",
                          "k = 3, Cluster 3")

modtable <- xtable(resum_prob,
                   digits = c(0, 3, 3, 3, 3, 3),
                   caption = "Distribution of the Frequencies of Assignment (Proportions) for Each Cl
                   label = "tab:frequency")
# cosmetics for table head (LaTeX slash (\) must be doubled inside a R chunk):
colnames(modtable) <- c("\\multicolumn{1}{c}{\\bf Minimum}",
```

```
                          "\\multicolumn{1}{c}{\\bf First Quartile}",
                          "\\multicolumn{1}{c}{\\bf Median}",
                          "\\multicolumn{1}{c}{\\bf Third Quartile}",
                          "\\multicolumn{1}{c}{\\bf Maximum}")

# format and print the table:
print(modtable,
      table.placement = "ht",
      size = "footnotesize",
      include.rownames = TRUE,
      include.colnames = TRUE,
      floating = TRUE,                  # whether \begin{table} should be created (TRUE) or not (FALSE
      sanitize.text.function = force)   # important to treat content of columns as latex function
```

	Minimum	First Quartile	Median	Third Quartile	Maximum
k = 2, Cluster 1	0.520	1.000	1.000	1.000	1.000
k = 2, Cluster 2	0.540	1.000	1.000	1.000	1.000
k = 3, Cluster 1	0.600	1.000	1.000	1.000	1.000
k = 3, Cluster 2	0.500	1.000	1.000	1.000	1.000
k = 3, Cluster 3	0.540	1.000	1.000	1.000	1.000

Table 1: Distribution of the Frequencies of Assignment (Proportions) for Each Cluster (for k = 2 and k = 3)

Resposta: Per fer la primera taula hem utilitzat la funció 'summary()' per als diferents valors de 'k', i utilitzant la plantilla per a fer taules proporcionada pel professorat, hem mostrat aquest resultat.

```
descr <- as.data.frame(res2$summarybycluster)

taula3 <- data.frame(Variable = gsub("_", "\\\\_", rownames(descr)),
                     Missing = as.numeric(descr[["%miss."]]),
                     Cluster1 = as.numeric(descr[["mean (cl.1)"]]),
                     Cluster2 = as.numeric(descr[["mean (cl.2)"]]),
                     check.names = FALSE
)

n <- res2$clusterssize[, "50%"] # Valor del nombre medià dels clusters

modtable <- xtable(taula3,
                   digits = c(0, 0, 2, 2, 2),
                   caption = "Description of the Variables (Mean Values) by Cluster",
                   label = "tab:descr")
# cosmetics for table head (LaTeX slash (\) must be doubled inside a R chunk):
colnames(modtable) <- c("\\multicolumn{1}{c}{\\bf Variable}",
                        "\\multicolumn{1}{c}{\\bf \\% of Subjects With Missing Values}",
                        paste0("\\multicolumn{1}{c}{\\bf Cluster 1 (n = ", n[1], ")}"), #paste0 per c
                        paste0("\\multicolumn{1}{c}{\\bf Cluster 2 (n = ", n[2], ")}"))

# format and print the table:
print(modtable,
      table.placement = "ht",
      size = "footnotesize",
      include.rownames = FALSE,
      include.colnames = TRUE,
      floating = TRUE,                  # whether \begin{table} should be created (TRUE) or not (FALSE
      sanitize.text.function = force)   # important to treat content of columns as latex function
```

Resposta: Per a la segona taula no hem inclós els 'F values

Variable	% of Subjects With Missing Values	Cluster 1 (n = 186)	Cluster 2 (n = 156)
activityv1	1.17	35.80	60.79
fevp_postv1	0.00	62.74	39.12
fvcp_postv1	0.00	79.93	63.65
iam_chv1	0.88	0.12	0.08
ic_prepv1	5.56	72.84	52.43
quo_calcpostv1	0.00	58.83	45.89
rv_prepv1	7.89	138.51	184.77
rv_tlcv1	7.89	50.89	63.19
sin15_v1	1.17	0.09	0.09
vas2_v1	1.17	4.18	6.49
vc_prepv1	5.56	77.48	60.45
impactsv1	1.17	18.61	35.81
po2_v1	3.22	77.39	69.67
graudisn_v1	1.17	1.91	3.27
vas1_v1	1.17	2.00	3.98
cccs1_v0	1.46	0.15	0.18
index_barthel2v1	2.63	100.00	98.15

Table 2: Description of the Variables (Mean Values) by Cluster